PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

TeamForge - Skill-Based Hackathon & Project Team Matchmaker

Student / Team Information

Team Name: *Team #:*	*TEAMFORGE* *DSCPP-III-2026-T238*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Mukul Fartiyal - 2510370466* *mukulfartiyal77@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Anubhav Bisht - 2510370178* *anubhav6875@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Siddhant Singh - 2510380233* *siddhantsingh1890@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Mehul - 2510370093* *mehul.200702@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Every time a hackathon, PBL, or project is announced, students usually start with the people they already know. A few messages are sent in a class or WhatsApp group and a team is formed quickly. The problem is that this does not always mean the team has the right mix of skills. A group can end up with several people working in the same area while nobody is comfortable with UI/UX, testing, presentation, or another role the project needs.

The information needed to avoid this is already available in a simple form. Students know the skills they can contribute, what they want to learn, their experience level, and the time they are available. A project brief can also list the skills or roles it needs. What is usually missing is a practical way to compare these two sides before the team is finalized.

TeamForge is proposed to handle that comparison. A student creates a skill profile, while a project or hackathon defines its requirements. The system then retrieves suitable candidates, gives them a compatibility score, ranks them, and shows which skills a possible team already covers and which ones are still missing. The main idea is not to match people only because they have similar skills. It is to find people whose skills complement the rest of the team. This makes the team-formation process more structured while still leaving the final choice with the students.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

At present, students mostly solve this problem through informal methods. Someone posts in a WhatsApp, Discord, or class group, and people join based on who replies or who they already know. Existing friend groups also tend to stay together even when the next project needs a different mix of skills. In some cases, students manually compare a list of members with the project requirements, but that becomes difficult when there are many students and skills to consider.

General collaboration and profile tools can help people find groups or list skills, but they do not directly answer the question: which available students fit this particular project's requirements, and what skill gaps will remain after choosing them? TeamForge is aimed at this missing step. It takes the skills offered by students and the requirements of a project and turns them into a ranked shortlist with visible coverage and gaps. The first version is intentionally focused on this core matching problem rather than depending on external services.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

- Let a student create a profile containing skills offered, skills wanted, experience level, and availability.
- Let a project or hackathon define the required skills, roles, and team-size limit.
- Retrieve suitable candidates and rank them using skill coverage, complementarity, experience, availability, and team balance.
- Check a proposed team before finalization and show its covered skills and remaining gaps.
- Save profiles, project requirements, and team history locally so the information remains available after the application is closed.
- Use the DSA concepts from TCS-302 and the OOP concepts from TCS-307 as part of the actual application design.

Milestones: The first phase is to settle the class model, Qt shell, profile format, and requirement format. The next phase is to build the data-structure layer for skill indexing, scoring, and ranking. After that, team formation will be added with coverage checks, duplicate and size validation, and gap analysis. The final stage is to test the system with small hand-checked datasets, compare selected algorithms in the Algorithm Lab, and prepare the final demonstration.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

We are building TeamForge in C++ with Qt for the desktop interface. VS Code and Qt Creator will be used during development, and Git/GitHub will be used for version control. For the first version, data will be kept in structured local files instead of a cloud database so that the main work stays on the matching logic and the course concepts.

A hash map will index students by skill so that candidate retrieval does not require checking every profile. Sets can prevent the same skill from being stored more than once, while vectors hold ordered requirement lists. A priority queue will keep the strongest candidates or team combinations near the top. A graph will represent useful compatibility relationships between students. A BST or AVL tree can also be used for ordered lookup and for comparison with other search methods in the Algorithm Lab.

The matching score is intentionally transparent. Required-skill coverage contributes 40%, complementarity 25%, experience/level 15%, availability 10%, and team diversity/balance 10%. The score is meant to explain a ranking rather than hide the reason behind it.

The C++ design will use classes for students, skills, requirements, matches, and teams. Inheritance, abstract strategy classes, virtual functions, polymorphism, constructors, templates, operator overloading, exception handling, and file I/O will be used where they fit the design. This keeps the DSA and OOP parts connected to the actual application.

Project Approach (continued)

Team Task Distribution

Member	Role	Planned contribution
Mukul Fartiyal	Team / Integration Lead	Requirements, overall architecture, coordination, and keeping the project work aligned.
Anubhav Bisht	UI / Testing / Documentation Lead	Qt interface planning, test planning, and maintaining project documentation.
Siddhant Singh	C++ / OOP Lead	C++ class design and the overall application structure, including the OOP side of the system.
Mehul	DSA Lead	Matching logic, searching, hashing, ranking, and the main data-structure work.

Course Integration

Course	Concepts applied	Where used
Data Structures in C	Searching, hashing, queues/priority queues, trees, linked structures, sorting, and graphs.	Candidate retrieval, ranking, coverage checks, team data, and Algorithm Lab.
OOP with C++	Classes and objects, constructors, inheritance, abstract classes, virtual functions, polymorphism, templates, exceptions, and file handling.	Application structure, strategy selection, data models, validation, and local persistence.

Main design idea: a student profile and a project requirement are the two inputs. The data-structure layer retrieves and organizes possible matches. The scoring and priority-queue layer ranks them. The team module then checks the selected combination for coverage and missing skills.

Matching Score Weights

Factor	Weight	Use in TeamForge
Required-skill coverage	40%	Measures how many required skills are covered.
Complementarity	25%	Rewards candidates who fill a gap instead of duplicating an existing skill.
Experience / level	15%	Considers relevant experience or proficiency.
Availability	10%	Considers overlap with project time requirements.
Team diversity / balance	10%	Helps avoid excessive concentration of the same role or skill.

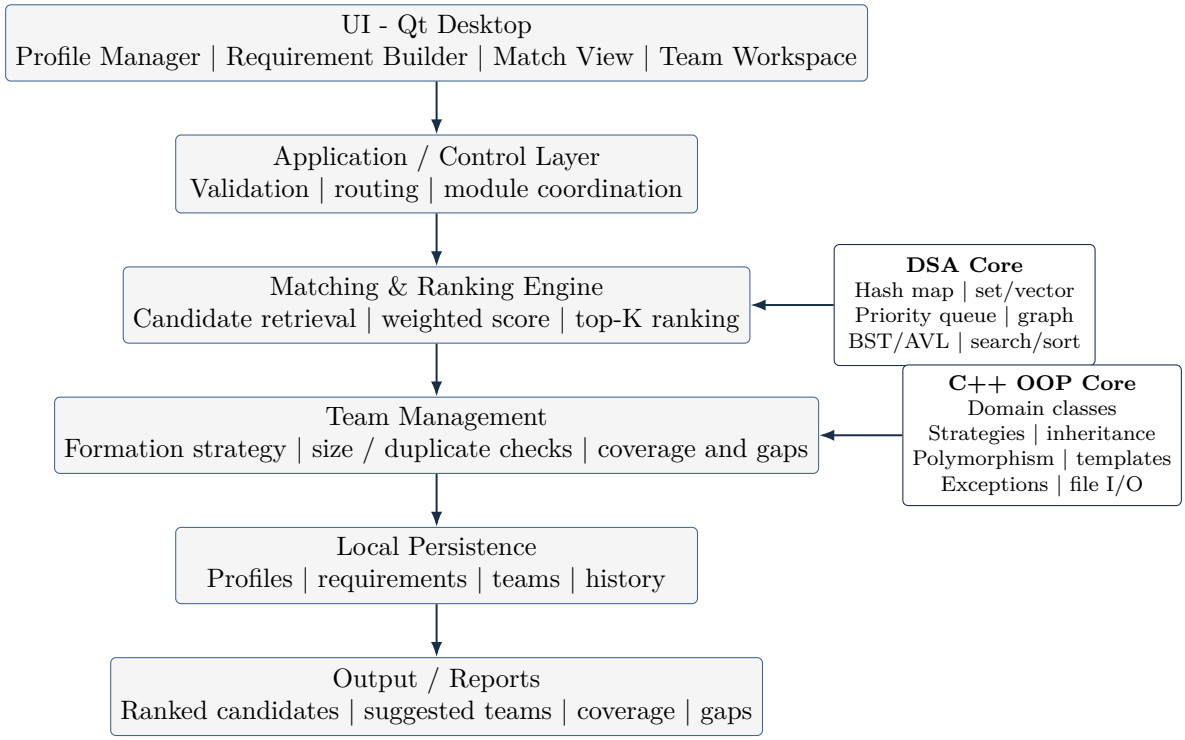
Project Approach (continued)

Development Roadmap

Week	Work	Expected deliverable
1	Project setup, class model, file format, basic Qt screens.	Working shell and profile persistence.
2	Arrays, stack, queue, deque, linked lists.	Profile history and request-queue structures.
3	Hashing, BST, AVL, heap, searching.	Skill/user lookup and ranking engine.
4	Sorting, match scoring, Algorithm Lab v1.	Ranked candidates and algorithm comparisons.
5	Graph engine, BFS, DFS, compatibility graph.	Skill graph and candidate traversal.
6	Team formation strategies, Union-Find, MST work.	Team creation and strategy switching.
7	Shortest paths, file blocks/indexing, B/B+ demonstration, exceptions, templates, STL.	Advanced modules and stable UI.
8	Testing, performance comparison, documentation, presentation, viva preparation.	Final PBL build and demonstration.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Working flow: Create Profile → Define Project Requirements → Retrieve Candidates → Score + Rank → Form Team → Check Coverage/Gaps → Save + Report.

The Qt interface is where students create profiles and project owners enter requirements. The application layer validates this information and passes it to the matching engine. Candidate retrieval uses the indexed skill data, after which the weighted score and ranking logic produce the shortlist. The team module then applies the selected formation strategy, checks team size and duplicate membership, and identifies covered and missing skills. The final information is written to local storage so that team history and previous work can be viewed later. The DSA and OOP parts are the working core of this flow.

Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The main outcome will be a C++/Qt desktop prototype that lets a student create a skill profile and lets a project or hackathon define its requirements. The system will retrieve relevant candidates, rank them using the matching score, and show the skill coverage and remaining gaps of a proposed team.
Along with the prototype, the team plans to deliver the source code, project documentation, the data-structure and OOP mapping, the matching-score description, and the Algorithm Lab work used to demonstrate selected searching, sorting, hashing, tree, and graph concepts. Local file persistence will keep profiles, requirements, and formed-team information available between sessions. A final demonstration will show the complete matching flow, strategy selection, coverage checking, and persistence.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words).

We assume that students enter reasonably accurate skill and availability information and that project requirements use identifiable skills or roles. The first version is intended to run locally without external APIs or cloud services. The number of candidates is assumed to be small enough for the matching process to be handled in memory while the data is also saved to files. The matching score is only a decision aid; it cannot guarantee that a team will work well together.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. Thareja, R. *Data Structures Using C*. Oxford University Press.
2. Balagurusamy, E. *Object Oriented Programming with C++*. McGraw Hill Education.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. 3rd ed., MIT Press.
4. Horowitz, E., & Sahni, S. *Fundamentals of Data Structures*. Galgotia.
5. Tenenbaum, A. M., Langsam, Y., & Augenstein, M. J. *Data Structures Using C and C++*. PHI.
6. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
7. Knuth, D. E., Morris, J. H., Jr., & Pratt, V. R. "Fast Pattern Matching in Strings." *SIAM Journal on Computing*, 6(2), 1977.
8. The Qt Company. *Qt 6 Documentation*, including Qt Widgets and Layouts.
9. cppreference.com. *C++ Language and Standard Library Reference*.